



Guia de instalação e configuração

Ubuntu Server 24.04 LTS em VPS na AWS (EC2) – passo a passo, sem Docker e sem build

Abrange: criação da instância EC2, preparação do Ubuntu, instalação em um comando ou manual (Node.js + PostgreSQL + systemd), Nginx + HTTPS (Let's Encrypt), e-mail (Amazon SES), primeiro acesso e migração do Clockify, backups no S3, atualização, RDS, segurança e solução de problemas

Repositório: [melhorartecnologia-arch/clockfy](https://github.com/melhorartecnologia-arch/clockfy)

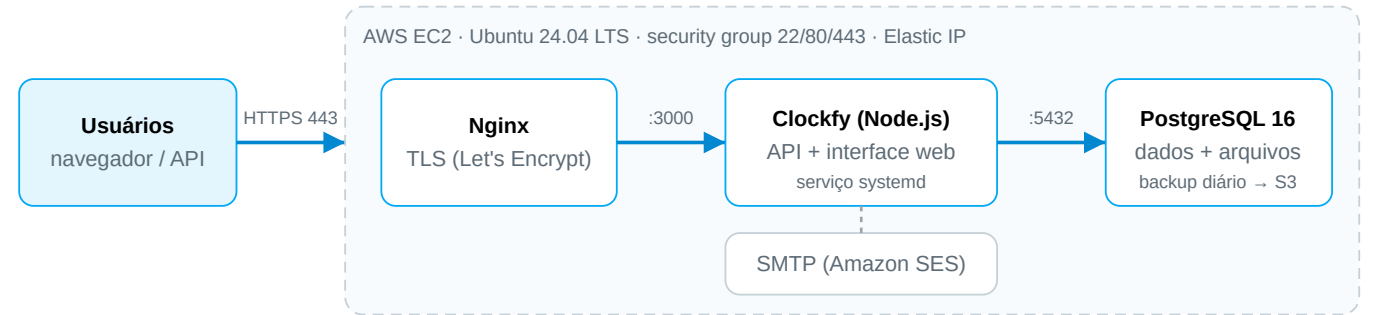
Data: 29/09/2026

Sumário

1	Visão geral e requisitos	9	Primeiro acesso e configuração inicial
2	Criar a instância EC2 (console AWS)	10	Backups e restauração
3	Conectar ao servidor por SSH	11	Atualizar o Clockfy
4	Preparar o Ubuntu 24.04	12	Operação: logs, monitoramento e comandos úteis
5	Instalação em um comando (instalador)	13	Opcional: banco gerenciado no Amazon RDS
6	Instalação manual passo a passo (sem Docker, sem build)	14	Segurança: lista de verificação
7	Nginx como proxy reverso e HTTPS (Let's Encrypt)	15	Solução de problemas
8	E-mail (SMTP) – convites, lembretes, relatórios e faturas	16	Apêndice: arquivos de implantação

1. Visão geral e requisitos

Este guia mostra, passo a passo, como colocar o **Clockfy** em produção em uma VPS **Ubuntu Server 24.04 LTS** na **AWS (EC2)**, **sem Docker e sem etapa de compilação**: a interface web já vem compilada no repositório (`web/dist`), então o servidor precisa apenas de Node.js e PostgreSQL. Um instalador (`deploy/install-ubuntu.sh`) faz toda a configuração em um comando; a seção 6 mostra os passos manuais equivalentes para quem preferir fazer à mão ou entender o que o instalador faz.



Dimensionamento sugerido

Cenário	Instância EC2	Disco	Observações
Até ~25 usuários	t3.small (2 vCPU, 2 GB)	30 GB gp3	Suficiente: não há compilação no servidor.
Até ~150 usuários	t3.medium (2 vCPU, 4 GB)	40 GB gp3	Confortável para relatórios grandes e importação do Clockify.
Acima disso / alta disponibilidade	t3.large + Amazon RDS PostgreSQL	40 GB + RDS	Banco gerenciado com backups automáticos (seção 13).

O que você precisa antes de começar

- Uma conta AWS com permissão para criar instâncias EC2 (e, opcionalmente, Route 53 e SES).
- Um domínio ou subdomínio para o sistema, ex.: `clockfy.suaempresa.com.br` (necessário para HTTPS e para os links de convite/relatórios).

- Acesso ao repositório do projeto: `https://github.com/melhorartecnologia-arch/clockfy` (branch `claude/zen-curie-y7y4bq` ou `main` após a integração).
- Um cliente SSH (Terminal no macOS/Linux; PowerShell ou PuTTY no Windows).

Convenções deste guia

Comandos começam com `$` e são executados no servidor como usuário `ubuntu` (com `sudo` quando indicado). Substitua `clockfy.suaempresa.com.br` pelo seu domínio e `SEU_IP` pelo IP elástico da instância. Os arquivos citados (`deploy/nginx-clockfy.conf` , `deploy/clockfy.service` , `deploy/backup.sh` ...) fazem parte do repositório.

2. Criar a instância EC2 (console AWS)

1. Acesse **console.aws.amazon.com** > **EC2** e escolha a região mais próxima dos usuários (ex.: **South America (São Paulo) sa-east-1**).
2. Clique em **Launch instance**. Em **Name**, informe `clockfy`.
3. Em **Application and OS Images**, selecione **Ubuntu Server 24.04 LTS (HVM), SSD Volume Type**, arquitetura **64-bit (x86)**.
4. Em **Instance type**, escolha `t3.small` (ou `t3.medium`, conforme a tabela da seção 1).
5. Em **Key pair (login)**, clique em **Create new key pair**: nome `clockfy-key`, tipo RSA, formato `.pem`. Baixe e guarde o arquivo – ele não pode ser baixado de novo.
6. Em **Network settings** > **Edit**: mantenha a VPC padrão, **Auto-assign public IP: Enable**. Crie um security group chamado `clockfy-sg` com as regras de entrada: **SSH (22)** apenas do seu IP (*My IP*), **HTTP (80)** de *Anywhere* (0.0.0.0/0) e **HTTPS (443)** de *Anywhere*. Não abra a porta 5432 nem a 3000.
7. Em **Configure storage**, defina **30 GiB** do tipo **gp3**.
8. Clique em **Launch instance** e aguarde o estado *Running* com *2/2 checks passed*.
9. **IP fixo**: em **EC2** > **Network & Security** > **Elastic IPs**, clique em **Allocate Elastic IP address** e depois em **Actions** > **Associate**, escolhendo a instância `clockfy`. Anote o IP (`SEU_IP`).
10. **DNS**: no seu provedor de DNS (ou **Route 53** > **Hosted zones**), crie um registro **A** para `clockfy.suaempresa.com.br` apontando para `SEU_IP` (TTL 300). Aguarde a propagação (normalmente minutos).

Boa prática

Ative o **Termination protection** (Actions > Instance settings > Change termination protection) e crie uma tag `Backup=true` para políticas de snapshot do EBS (AWS Backup ou Data Lifecycle Manager).

3. Conectar ao servidor por SSH

No macOS/Linux (Terminal) ou Windows (PowerShell):

```
chmod 400 ~/Downloads/clockfy-key.pem          # só no macOS/Linux
ssh -i ~/Downloads/clockfy-key.pem ubuntu@SEU_IP
```

No Windows com PuTTY, converta o `.pem` para `.ppk` com o PuTTYgen e use o usuário `ubuntu`. Na primeira conexão, confirme a impressão digital do host com **yes**.

Se a conexão falhar

Confira se a regra SSH do security group contém o seu IP atual (ele muda em redes domésticas/móveis), se o IP usado é o Elastic IP e se a permissão do arquivo `.pem` é 400.

4. Preparar o Ubuntu 24.04

4.1 Atualizações, fuso horário e hostname

```
sudo apt update && sudo apt -y upgrade
sudo timedatectl set-timezone America/Sao_Paulo
sudo hostnamectl set-hostname clockfy
sudo apt -y install git curl ufw fail2ban unattended-upgrades ca-certificates gnupg
sudo dpkg-reconfigure -plow unattended-upgrades      # escolha "Yes" para atualizações de segurança
automáticas
sudo reboot      # se o upgrade pedir reinicialização (reconecte após ~1 minuto)
```

4.2 Firewall (ufw) e proteção contra força bruta

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw --force enable && sudo ufw status verbose
sudo systemctl enable --now fail2ban
```

O ufw complementa o security group da AWS: as portas 3000 (aplicação) e 5432 (banco) ficam acessíveis apenas dentro do servidor. O instalador da seção 5 também aplica estas regras; executá-las antes não faz mal.

5. Instalação em um comando (instalador)

O script `deploy/install-ubuntu.sh` instala o Node.js 22 e o PostgreSQL 16, cria o banco e o usuário de serviço, clona o repositório (já com a interface compilada), gera o `.env` com um `JWT_SECRET` aleatório, aplica as migrações, registra o serviço `systemd`, configura o Nginx, emite o certificado HTTPS (se o domínio já apontar para o servidor), ativa o firewall e agenda o backup diário. Ele pode ser executado de novo sem problemas (é idempotente): reexecutar apenas atualiza o código e reinicia o serviço.

5.1 Executar

```
sudo apt-get install -y git
git clone https://github.com/melhorartecnologia-arch/clockfy.git /tmp/clockfy-src
sudo DOMAIN=clockfy.suaempresa.com.br LETSENCRYPT_EMAIL=voce@suaempresa.com.br BRANCH=main \
  bash /tmp/clockfy-src/deploy/install-ubuntu.sh
```

Enquanto o código estiver apenas na branch de desenvolvimento, use `BRANCH=claude/zen-curie-y7y4bq`. Se o repositório for privado, clone com um token: `https://TOKEN@github.com/melhorartecnologia-arch/clockfy.git` e passe o mesmo endereço em `REPO_URL=...`.

Variável	Padrão	Para que serve
DOMAIN	(vazio)	Domínio do sistema. Com ele, o Nginx responde pelo domínio e o certificado HTTPS é emitido. Sem ele, o acesso fica em <code>http://SEU_IP</code> .
LETSENCRYPT_EMAIL	(vazio)	E-mail para avisos de expiração do certificado.
BRANCH / REPO_URL	<code>main</code> / GitHub do projeto	Qual código instalar.
APP_DIR	<code>/opt/clockfy</code>	Pasta de instalação.
DB_PASSWORD	gerada	Senha do usuário <code>clockfy</code> no PostgreSQL (gravada no <code>.env</code>).
SKIP_NGINX=1 / SKIP_SSL=1	0	Pular a configuração do Nginx ou a emissão do certificado.

5.2 O que o instalador deixa pronto

Item	Onde
Código da aplicação (interface já compilada em <code>web/dist</code>)	<code>/opt/clockfy</code> , dono <code>clockfy</code>
Dependências de produção (só do servidor, sem Vite/React)	<code>/opt/clockfy/node_modules</code> (via <code>npm ci --omit=dev --workspace=server</code>)
Configuração	<code>/opt/clockfy/server/.env</code> (permissão 600)
Banco de dados	PostgreSQL local, banco <code>clockfy</code> , usuário <code>clockfy</code>
Serviço	<code>systemctl status clockfy</code> · logs: <code>journalctl -u clockfy -f</code>
Proxy e HTTPS	<code>/etc/nginx/sites-available/clockfy</code> + Certbot (renovação automática)
Backup diário 03:00	<code>/var/backups/clockfy</code> (14 dias), script <code>deploy/backup.sh</code>
Firewall	ufw: 22, 80 e 443; fail2ban ativo

5.3 Verificar

```
systemctl status clockfy --no-pager
curl -s http://127.0.0.1:3000/health      # {"status":"ok",...}
sudo journalctl -u clockfy -n 30 --no-pager  # "[clockfy] listening on http://localhost:3000"
```

Abra `https://clockfy.suaempresa.com.br` (ou `http://SEU_IP` sem domínio) e siga para a seção 8 (e-mail) e 9 (primeiro acesso). Se o certificado não foi emitido porque o DNS ainda não propagou, execute depois: `sudo certbot --nginx -d clockfy.suaempresa.com.br --redirect` e ajuste `APP_URL` para `https` no `.env`.

6. Instalação manual passo a passo (sem Docker, sem build)

Equivalente ao instalador, para quem prefere executar cada etapa. Não há compilação: a pasta `web/dist` versionada é servida diretamente pelo servidor Node.js.

6.1 Node.js 22

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt -y install nodejs
node -v && npm -v          # v22.x
```

6.2 PostgreSQL 16

```
sudo apt -y install postgresql postgresql-contrib
sudo systemctl enable --now postgresql
sudo -u postgres psql -c "CREATE USER clockfy WITH PASSWORD 'TROQUE_ESTA_SENHA';"
sudo -u postgres psql -c "CREATE DATABASE clockfy OWNER clockfy ENCODING 'UTF8';"
```

6.3 Usuário de serviço e código

```
sudo adduser --system --group --home /opt/clockfy --shell /bin/bash clockfy
sudo -u clockfy git clone -b main https://github.com/melhorartecnologia-arch/clockfy.git /opt/clockfy
ls /opt/clockfy/web/dist/index.html      # interface já compilada
```

6.4 Dependências de produção (somente servidor)

```
cd /opt/clockfy
sudo -u clockfy npm ci --omit=dev --workspace=server
```

Este comando instala apenas as bibliotecas do servidor (Express, pg, etc.), sem Vite ou React – nada é compilado.

6.5 Configuração (.env em server/)

```
sudo -u clockfy cp /opt/clockfy/.env.example /opt/clockfy/server/.env
openssl rand -hex 32                      # use o resultado em JWT_SECRET
sudo -u clockfy nano /opt/clockfy/server/.env
sudo chmod 600 /opt/clockfy/server/.env
```

```
PORT=3000
NODE_ENV=production
DATABASE_URL=postgres://clockfy:TROQUE_ESTA_SENHA@localhost:5432/clockfy
JWT_SECRET=valor-gerado-pelo-openssl
APP_URL=https://clockfy.suaempresa.com.br
SMTP_HOST=
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=
SMTP_PASS=
SMTP_FROM=Clockfy <no-reply@suaempresa.com.br>
```

6.6 Migrações e serviço systemd

```
cd /opt/clockfy/server && sudo -u clockfy node src/cli/migrate.js      # cria as tabelas
sudo cp /opt/clockfy/deploy/clockfy.service /etc/systemd/system/clockfy.service
sudo systemctl daemon-reload
sudo systemctl enable --now clockfy
sudo systemctl status clockfy --no-pager
curl -s http://localhost:3000/health
```

O serviço reinicia automaticamente em caso de falha e após reboot. Continue na seção 7 (Nginx e HTTPS).

7. Nginx como proxy reverso e HTTPS (Let's Encrypt)

7.1 Instalar e configurar o Nginx

```
sudo apt -y install nginx
sudo cp /opt/clockfy/deploy/nginx-clockfy.conf /etc/nginx/sites-available/clockfy
sudo sed -i 's/clockfy.exemplo.com.br/clockfy.suaempresa.com.br/g' /etc/nginx/sites-available/clockfy
sudo ln -s /etc/nginx/sites-available/clockfy /etc/nginx/sites-enabled/clockfy
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx
```

Teste em `http://clockfy.suaempresa.com.br` – a tela de login deve aparecer (ainda sem cadeado).

7.2 Certificado HTTPS gratuito com o Certbot

```
sudo snap install core && sudo snap refresh core
sudo snap install --classic certbot
sudo ln -sf /snap/bin/certbot /usr/bin/certbot
sudo certbot --nginx -d clockfy.suaempresa.com.br --redirect -m seu-email@suaempresa.com.br --agree-tos -n
sudo certbot renew --dry-run # testa a renovação automática (o snap agenda a renovação sozinho)
```

O Certbot altera o arquivo do Nginx para escutar na 443 com o certificado e redirecionar HTTP para HTTPS. A renovação ocorre automaticamente a cada ~60 dias.

Confira o APP_URL

Depois do HTTPS, garanta que `APP_URL=https://clockfy.suaempresa.com.br` em `/opt/clockfy/server/.env` e reinicie a aplicação (`sudo systemctl restart clockfy`). O instalador da seção 5 já faz isso quando emite o certificado. Os links de convite, de relatórios compartilhados e do quiosque usam esse valor.

8. E-mail (SMTP) – convites, lembretes, relatórios e faturas

Sem SMTP o sistema funciona normalmente (os links de convite podem ser copiados na tela de equipe e as notificações ficam no app), mas e-mails de convite, lembretes, alertas, relatórios agendados e faturas só são enviados com um servidor SMTP configurado.

Opção 1 – Amazon SES (recomendado na AWS)

1. No console **Amazon SES** (mesma região ou *us-east-1*), em **Identities > Create identity**, verifique o seu domínio (adicione os registros DKIM no DNS) ou um endereço de e-mail.
2. Em **SMTP settings > Create SMTP credentials**, gere o usuário e a senha SMTP (guarde-os; são diferentes das chaves IAM).
3. Enquanto a conta estiver em *sandbox*, só é possível enviar para endereços verificados; solicite a saída do sandbox em **Account dashboard > Request production access**.

4. Configure no `.env` :

```
SMTP_HOST=email-smtp.sa-east-1.amazonaws.com    # use o endpoint da região do SES
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=AKIAxxxxxxxxxxxxxxxxxxxxxx
SMTP_PASS=BFxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
SMTP_FROM=Clockfy <no-reply@suaempresa.com.br>
```

Opção 2 – Google Workspace / Gmail

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=no-reply@suaempresa.com.br
SMTP_PASS=senha-de-app-de-16-caracteres          # Conta Google > Segurança > Senhas de app (requer 2FA)
SMTP_FROM=Clockfy <no-reply@suaempresa.com.br>
```

Após alterar `/opt/clockfy/server/.env`, execute `sudo systemctl restart clockfy` e teste convidando um membro em **Equipe > Convidar membros**. Falhas de envio aparecem em `journalctl -u clockfy` com o prefixo `[mail]`.

9. Primeiro acesso e configuração inicial

1. Abra `https://clockfy.suaempresa.com.br` e clique em **Criar conta**. O primeiro usuário vira proprietário/administrador do workspace criado.
2. Em **Configurações > Geral**: nome do workspace, moeda (ex.: BRL), taxa horária e custo padrão, dias úteis e capacidade diária, formatos.
3. Em **Configurações > Permissões** e **Campos obrigatórios**: defina quem pode criar projetos/etiquetas/tarefas, quem vê taxas e o que é obrigatório ao registrar tempo.
4. Em **Configurações > Funcionalidades**: ligue/desligue aprovações, folgas, agenda, despesas, faturas e quiosque.
5. Cadastre **Clientes**, **Projetos** (com tarefas e estimativas) e **Etiquetas** – ou importe tudo do Clockify (próximo passo).
6. **Migração**: em **Administração > Importar do Clockify**, informe a chave de API do Clockify (Profile settings > API), escolha o workspace de origem e execute. Reexecute com “desde” para trazer registros novos até desligar o Clockify.
7. Convide a equipe em **Equipe > Convidar membros** e defina papéis (admin, gerente de equipe, gerente de projeto).
8. Gere chaves de API em **Perfil > Chaves de API** para integrações; aponte-as para `https://clockfy.suaempresa.com.br/api/v1`.

Dados de demonstração (opcional, só em ambientes de teste)

```
cd /opt/clockfy/server && sudo -u clockfy node src/cli/seed.js cria o usuário
admin@clockfy.local / admin123 com projetos e registros de exemplo. Não use em produção.
```

10. Backups e restauração

Todos os dados (inclusive recibos e logotipos) ficam no PostgreSQL, portanto um dump do banco é um backup completo. O script `deploy/backup.sh` mantém 14 dias de arquivos e pode enviar para o S3. O instalador já agenda a execução diária às 03:00; os comandos abaixo servem para conferir ou configurar manualmente.

10.1 Backup diário automático

```
sudo mkdir -p /var/backups/clockfy
sudo /opt/clockfy/deploy/backup.sh # teste manual
sudo crontab -e # adicione a linha abaixo (todo dia às 03:00)
0 3 * * * /opt/clockfy/deploy/backup.sh >> /var/log/clockfy-backup.log 2>&1
```

10.2 Enviar para o Amazon S3 (recomendado)

1. Crie um bucket (ex.: `suaempresa-backups`) com *Block public access* ativado e uma regra de ciclo de vida para expirar objetos após 90 dias.
2. Crie uma **IAM role** para EC2 com permissão `s3:PutObject` no bucket e associe-a à instância (Actions > Security > Modify IAM role) – sem chaves no servidor.
3. Instale a CLI e defina o bucket no cron:

```
sudo snap install aws-cli --classic
0 3 * * * S3_BUCKET=s3://suaempresa-backups/clockfy /opt/clockfy/deploy/backup.sh >> /var/log/clockfy-backup.log 2>&1
```

10.3 Restaurar

```
ls -lh /var/backups/clockfy/
sudo /opt/clockfy/deploy/restore.sh /var/backups/clockfy/clockfy-20260929-030000.sql.gz
```

Complementarmente, ative snapshots automáticos do volume EBS (AWS Backup ou Data Lifecycle Manager) para recuperação do servidor inteiro.

11. Atualizar o Clockfy

```
sudo /opt/clockfy/deploy/update.sh
```

O script faz um backup, executa `git pull`, reinstala as dependências de produção (sem compilar nada – a interface atualizada já vem em `web/dist`) e reinicia o serviço; as migrações novas são aplicadas automaticamente na inicialização. Para voltar a uma versão anterior: `sudo -u clockfy git -C /opt/clockfy checkout <commit>`, `sudo systemctl restart clockfy` e, se necessário, restaure o backup (seção 10.3).

12. Operação: logs, monitoramento e comandos úteis

Tarefa	Comando
Ver logs em tempo real	<code>sudo journalctl -u clockfy -f</code>
Reiniciar / parar / iniciar	<code>sudo systemctl restart stop start clockfy</code>
Status do serviço	<code>systemctl status clockfy</code>
Acessar o banco	<code>sudo -u postgres psql -d clockfy</code>
Alterar configuração	editar <code>/opt/clockfy/server/.env</code> e <code>sudo systemctl restart clockfy</code>
Verificação de saúde	<code>curl -s https://clockfy.suaempresa.com.br/health</code>
Espaço em disco	<code>df -h</code> · backups em <code>/var/backups/clockfy</code>
Certificado HTTPS	<code>sudo certbot certificates</code> · renovação: <code>sudo certbot renew --dry-run</code>
Reexecutar o instalador (atualiza e corrige a configuração)	<code>sudo DOMAIN=... bash /opt/clockfy/deploy/install-ubuntu.sh</code>

Para alertas de indisponibilidade, use um monitor externo (Amazon CloudWatch Synthetics, UptimeRobot ou similar) apontando para `/health`. Para métricas do servidor, o **CloudWatch agent** pode ser instalado opcionalmente.

13. Opcional: banco gerenciado no Amazon RDS

1. Em **RDS › Create database**: PostgreSQL 16, template *Production* ou *Dev/Test*, instância `db.t4g.micro` ou superior, armazenamento gp3 20 GB, **Public access**: **No**, mesma VPC da EC2.
2. No security group do RDS, libere a porta 5432 apenas para o security group `clockfy-sg` da EC2.
3. Crie o banco `clockfy` (nome inicial do banco na criação) e anote endpoint, usuário e senha.
4. Configure `DATABASE_URL=postgres://usuario:senha@ENDPOINT.rds.amazonaws.com:5432/clockfy?sslmode=no-verify` (o RDS exige TLS por padrão) e reinicie a aplicação.
5. O PostgreSQL local pode ser desativado (`sudo systemctl disable --now postgresql`). Os backups passam a ser feitos pelos snapshots automáticos do RDS.

14. Segurança: lista de verificação

- Acesso SSH somente por chave e restrito ao seu IP no security group; `fail2ban` ativo.
- Portas 3000 e 5432 fechadas para a internet (ufw + security group); o PostgreSQL escuta apenas em localhost.
- `JWT_SECRET` longo e aleatório; senha do banco alterada; arquivo `.env` com permissão 600 (`chmod 600 .env`).
- HTTPS obrigatório (Certbot com redirecionamento) e `APP_URL` em https.

- Atualizações de segurança automáticas do Ubuntu (`unattended-upgrades`) e atualização periódica do Clockfy.
- Backups diários testados (restauração) e cópia fora do servidor (S3) + snapshots EBS.
- Na aplicação: apenas administradores gerenciam faturas, configurações e importação; revise as permissões em **Configurações > Permissões**; revogue chaves de API não usadas em **Perfil > Chaves de API**.

15. Solução de problemas

Sintoma	Causa provável	Como resolver
502 Bad Gateway no navegador	A aplicação não está rodando na porta 3000.	<code>systemctl status clockfy</code> e <code>journalctl -u clockfy -n 50</code> ; normalmente é erro no <code>.env</code> (banco) – corrija e reinicie.
Página não abre, nem HTTP	Security group sem 80/443, ufw bloqueando ou DNS não propagado.	<code>sudo ufw status</code> ; regras do security group; <code>dig clockfy.suaempresa.com.br</code> deve retornar SEU_IP.
<code>ECONNREFUSED / password authentication failed</code> nos logs	<code>DATABASE_URL</code> incorreta ou PostgreSQL parado.	Confira usuário/senha/host no <code>.env</code> ; <code>systemctl status postgresql</code> .
Instalador para em <code>web/dist</code> não encontrado	A branch clonada não contém a interface compilada.	Use a branch correta (<code>BRANCH=main</code> ou <code>claude/zen-curie-y7y4bq</code>); em desenvolvimento, gere e versiona o <code>web/dist</code> com <code>npm run build</code> .
<code>npm ci</code> falha por versão do Node	Node.js antigo (Ubuntu traz o 18).	Instale o Node 22 pelo NodeSource (seção 6.1) e repita.
Certbot falha na validação	DNS ainda não aponta para o servidor ou porta 80 fechada.	Aguarde a propagação, confira o registro A e o security group, repita o comando.
E-mails não chegam	SMTP não configurado, credenciais erradas ou SES em sandbox.	Procure <code>[mail]</code> nos logs; verifique remetente/domínio no SES; teste com outro provedor.
API responde 401	Chave de API ausente/inválida ou token expirado.	Gere nova chave em Perfil > Chaves de API e envie no header <code>X-Api-Key</code> .
Links de convite/relatório com endereço errado	<code>APP_URL</code> desatualizado.	Ajuste no <code>.env</code> e reinicie a aplicação.
Disco cheio	Backups acumulados ou logs.	Reduza <code>KEEP_DAYS</code> do backup; <code>sudo journalctl --vacuum-size=200M</code> ; aumente o volume EBS.

16. Apêndice: arquivos de implantação

deploy/install-ubuntu.sh (instalador)

```
#!/usr/bin/env bash
# =====
# Clockfy - instalador para Ubuntu Server 24.04 LTS (sem Docker, sem build)
#
# Instala Node.js 22, PostgreSQL 16, Nginx, cria o banco, o usuário de serviço,
# clona o repositório (a interface já vem compilada em web/dist), gera o .env,
# registra o serviço systemd e, se DOMAIN for informado, configura o Nginx e o
# certificado HTTPS (Let's Encrypt). Pode ser executado várias vezes (idempotente).
#
# Uso (como root ou com sudo):
#   sudo DOMAIN=clockfy.suaempresa.com.br LETSENCRYPT_EMAIL=voce@suaempresa.com.br \
#       bash deploy/install-ubuntu.sh
#
# Variáveis opcionais:
#   REPO_URL      (padrão: https://github.com/melhorartecnologia-arch/clockfy.git)
#   BRANCH        (padrão: main; use claude/zen-curie-y7y4bq até a integração)
#   APP_DIR       (padrão: /opt/clockfy)
#   DB_PASSWORD   (padrão: gerada automaticamente e gravada no .env)
#   SKIP_NGINX=1  não configura o Nginx   SKIP_SSL=1  não emite certificado
# =====
set -euo pipefail

REPO_URL=${REPO_URL:-https://github.com/melhorartecnologia-arch/clockfy.git}
BRANCH=${BRANCH:-main}
APP_DIR=${APP_DIR:-/opt/clockfy}
DOMAIN=${DOMAIN:-}
LETSENCRYPT_EMAIL=${LETSENCRYPT_EMAIL:-}
DB_NAME=${DB_NAME:-clockfy}
DB_USER=${DB_USER:-clockfy}
DB_PASSWORD=${DB_PASSWORD:-}
SERVICE_USER=clockfy
PORT=${PORT:-3000}
SKIP_NGINX=${SKIP_NGINX:-0}
SKIP_SSL=${SKIP_SSL:-0}

log() { echo -e "\033[1;34m[clockfy]\033[0m $*"; }
ok() { echo -e "\033[1;32m[ok]\033[0m $*"; }
fail() { echo -e "\033[1;31m[erro]\033[0m $*" >&2; exit 1; }

[ "$(id -u)" -eq 0 ] || fail "execute como root: sudo bash deploy/install-ubuntu.sh"
grep -qi ubuntu /etc/os-release || fail "este instalador foi feito para Ubuntu (24.04 LTS)"
export DEBIAN_FRONTEND=noninteractive

# ----- 1. pacotes
log "Atualizando pacotes do sistema"
apt-get update -qq || echo "[aviso] apt-get update terminou com avisos"
apt-get install -y -qq curl git ca-certificates gnupg ufw fail2ban unattended-upgrades openssl cron
sudo >/dev/null
ok "pacotes básicos"

if ! command -v node >/dev/null 2>&1 || [ "$(node -v | cut -d. -f1 | tr -d v)" -lt 20 ]; then
    log "Instalando Node.js 22 (NodeSource)"
    curl -fsSL https://deb.nodesource.com/setup_22.x | bash - >/dev/null
    apt-get install -y -qq nodejs >/dev/null
fi
NODE_BIN=$(command -v node); NPM_BIN=$(command -v npm)
ok "Node.js ${NODE_BIN} / npm ${NPM_BIN} (${NODE_BIN})"
# executa comandos como o usuário de serviço mantendo PATH e proxy (se houver)
```

```

as_service() { sudo -u "$SERVICE_USER" env PATH="$PATH" HOME="$APP_DIR"
${HTTPS_PROXY:+HTTPS_PROXY="$HTTPS_PROXY"} ${HTTP_PROXY:+HTTP_PROXY="$HTTP_PROXY"}
${NODE_EXTRA_CA_CERTS:+NODE_EXTRA_CA_CERTS="$NODE_EXTRA_CA_CERTS"} bash -c "$1"; }

if ! command -v psql >/dev/null 2>&1; then
    log "Instalando PostgreSQL"
    apt-get install -y -qq postgresql postgresql-contrib >/dev/null
fi
systemctl enable --now postgresql >/dev/null 2>&1 || true
ok "PostgreSQL $(psql --version | awk '{print $3}')"

# ----- 2. usuário e código
if ! id -u "$SERVICE_USER" >/dev/null 2>&1; then
    adduser --system --group --home "$APP_DIR" --shell /bin/bash "$SERVICE_USER" >/dev/null
fi
mkdir -p "$APP_DIR"
if [ -d "$APP_DIR/.git" ]; then
    log "Atualizando código em $APP_DIR (branch $BRANCH)"
    sudo -u "$SERVICE_USER" git -C "$APP_DIR" fetch --quiet origin "$BRANCH"
    sudo -u "$SERVICE_USER" git -C "$APP_DIR" checkout --quiet "$BRANCH"
    sudo -u "$SERVICE_USER" git -C "$APP_DIR" pull --quiet --ff-only origin "$BRANCH"
else
    log "Clonando $REPO_URL (branch $BRANCH) em $APP_DIR"
    chown "$SERVICE_USER:$SERVICE_USER" "$APP_DIR"
    sudo -u "$SERVICE_USER" git clone --quiet --branch "$BRANCH" "$REPO_URL" "$APP_DIR"
fi
chown -R "$SERVICE_USER:$SERVICE_USER" "$APP_DIR"
[ -f "$APP_DIR/web/dist/index.html" ] || fail "web/dist não encontrado no repositório - a interface
precisa estar compilada (npm run build) e versionada"
ok "código pronto"

log "Instalando dependências de produção (somente servidor)"
as_service "cd '$APP_DIR' && '$NPM_BIN' ci --omit=dev --workspace=server --no-audit --no-fund --
loglevel=error" || true
[ -d "$APP_DIR/node_modules/pg" ] && [ -d "$APP_DIR/node_modules/express" ] || fail "npm ci não
instalou as dependências (veja $APP_DIR/.npm/_logs); verifique o acesso à internet e tente de novo"
ok "dependências instaladas"

# ----- 3. banco
ENV_FILE="$APP_DIR/server/.env"
if [ -f "$ENV_FILE" ] && grep -q '^DATABASE_URL=' "$ENV_FILE"; then
    DB_PASSWORD=$(grep '^DATABASE_URL=' "$ENV_FILE" | sed -E 's#^DATABASE_URL=postgres://[^:]+:
([^\@]+)\@.*#\1#')
fi
[ -n "$DB_PASSWORD" ] || DB_PASSWORD=$(openssl rand -hex 16)
if ! sudo -u postgres psql -tAc "SELECT 1 FROM pg_roles WHERE rolname='$DB_USER'" | grep -q 1; then
    sudo -u postgres psql -qc "CREATE USER $DB_USER WITH PASSWORD '$DB_PASSWORD';"
else
    sudo -u postgres psql -qc "ALTER USER $DB_USER WITH PASSWORD '$DB_PASSWORD';"
fi
if ! sudo -u postgres psql -tAc "SELECT 1 FROM pg_database WHERE datname='$DB_NAME'" | grep -q 1; then
    sudo -u postgres psql -qc "CREATE DATABASE $DB_NAME OWNER $DB_USER ENCODING 'UTF8';"
fi
ok "banco $DB_NAME (usuário $DB_USER)"

# ----- 4. .env
if [ ! -f "$ENV_FILE" ]; then
    log "Gerando $ENV_FILE"
    APP_URL_VALUE="http://$(hostname -I | awk '{print $1}'):5432/$DB_NAME"
    [ -n "$DOMAIN" ] && APP_URL_VALUE="http://$DOMAIN"
    cat > "$ENV_FILE" <<ENV
PORT=$PORT
NODE_ENV=production
DATABASE_URL=postgres://$DB_USER:$DB_PASSWORD@localhost:5432/$DB_NAME
JWT_SECRET=$(openssl rand -hex 32)

```

```

APP_URL=$APP_URL_VALUE
# E-mail (opcional): sem SMTP os e-mails são apenas registrados no log
SMTP_HOST=
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=
SMTP_PASS=
SMTP_FROM="Clockfy <no-reply@${DOMAIN:-localhost}>"
RATE_LIMIT_PER_SECOND=50
SCHEDULER_ENABLED=true
MAX_UPLOAD_BYTES=10485760

ENV
  chown "$SERVICE_USER:$SERVICE_USER" "$ENV_FILE"; chmod 600 "$ENV_FILE"
  ok ".env criado"
else
  ok ".env existente mantido"
fi

log "Aplicando migrações do banco"
as_service "cd '$APP_DIR/server' && '$NODE_BIN' src/cli/migrate.js" | tail -1

# ----- 5. systemd
install -m 644 "$APP_DIR/deploy/clockfy.service" /etc/systemd/system/clockfy.service
sed -i -e "s#/opt/clockfy#$APP_DIR#g" -e "s#ExecStart=/usr/bin/node#ExecStart=$NODE_BIN#"
/etc/systemd/system/clockfy.service
systemctl daemon-reload
systemctl enable clockfy >/dev/null 2>&1
systemctl restart clockfy
sleep 3
if curl -fsS "http://127.0.0.1:$PORT/health" >/dev/null; then ok "serviço clockfy ativo na porta $PORT"; else journalctl -u clockfy -n 30 --no-pager; fail "o serviço não respondeu em /health"; fi

# ----- 6. nginx
if [ "$SKIP_NGINX" != "1" ]; then
  apt-get install -y -qq nginx >/dev/null
  SERVER_NAME=${DOMAIN:-_}
  sed -e "s/clockfy.exemplo.com.br/$SERVER_NAME/" -e "s#127.0.0.1:3000#127.0.0.1:$PORT#"
"$APP_DIR/deploy/nginx-clockfy.conf" > /etc/nginx/sites-available/clockfy
ln -sf /etc/nginx/sites-available/clockfy /etc/nginx/sites-enabled/clockfy
rm -f /etc/nginx/sites-enabled/default
nginx -t >/dev/null && systemctl enable --now nginx >/dev/null 2>&1 && systemctl reload nginx
ok "nginx configurado (server_name $SERVER_NAME)"
if [ -n "$DOMAIN" ] && [ "$SKIP_SSL" != "1" ]; then
  log "Emitindo certificado HTTPS para $DOMAIN"
  if ! command -v certbot >/dev/null 2>&1; then snap install core >/dev/null 2>&1 || true; snap
install --classic certbot >/dev/null; ln -sf /snap/bin/certbot /usr/bin/certbot; fi
  if certbot --nginx -d "$DOMAIN" --redirect -n --agree-tos ${LETSENCRYPT_EMAIL:+-m
"$LETSENCRYPT_EMAIL"} ${LETSENCRYPT_EMAIL:---register-unsafely-without-email}; then
    sed -i "s#^APP_URL=. *#APP_URL=https://$DOMAIN#" "$ENV_FILE"; systemctl restart clockfy
    ok "HTTPS ativo em https://$DOMAIN"
  else
    echo "[aviso] certbot falhou (DNS ainda não aponta para este servidor?). Repita depois: certbot -
-nginx -d $DOMAIN --redirect"
  fi
fi
fi

# ----- 7. firewall
if ufw allow OpenSSH >/dev/null 2>&1 && ufw allow 80/tcp >/dev/null 2>&1 && ufw allow 443/tcp
>/dev/null 2>&1 && ufw --force enable >/dev/null 2>&1; then
  ok "firewall ufw ativo (22, 80, 443)"
else
  echo "[aviso] não foi possível ativar o ufw; garanta o firewall pelo security group da AWS"
fi
systemctl enable --now fail2ban >/dev/null 2>&1 || true

```

```
# ----- 8. backup
mkdir -p /var/backups/clockfy
( crontab -l 2>/dev/null | grep -v 'deploy/backup.sh' || true; echo "0 3 * * *
$APP_DIR/deploy/backup.sh >> /var/log/clockfy-backup.log 2>&1" ) | crontab -
ok "backup diário agendado (03:00) em /var/backups/clockfy"

echo
echo "=====
echo " Clockfy instalado!"
echo "   Endereço : $(grep '^APP_URL=' "$ENV_FILE" | cut -d= -f2-)"
echo "   Código   : $APP_DIR      .env: $ENV_FILE"
echo "   Serviço   : systemctl status clockfy   logs: journalctl -u clockfy -f"
echo "   Atualizar: sudo $APP_DIR/deploy/update.sh"
echo "   Abra o endereço no navegador e clique em 'Criar conta' (o primeiro usuário vira administrador)."
echo "=====
```

deploy/update.sh

```
#!/usr/bin/env bash
# Atualiza o Clockfy para a versão mais recente do repositório (sem build: web/dist já vem compilado).
# Uso: sudo /opt/clockfy/deploy/update.sh
set -euo pipefail
APP_DIR=${CLOCKFY_DIR:-/opt/clockfy}
SERVICE_USER=$(stat -c %U "$APP_DIR")
"$APP_DIR/deploy/backup.sh" || echo "[aviso] backup falhou; continuando"
sudo -u "$SERVICE_USER" git -C "$APP_DIR" pull --ff-only
sudo -u "$SERVICE_USER" bash -c "cd '$APP_DIR' && npm ci --omit=dev --workspace=server --no-audit --no-fund --loglevel=error"
systemctl restart clockfy
sleep 3 && curl -fsS http://127.0.0.1:3000/health >/dev/null && echo "Clockfy atualizado e ativo" || {
journalctl -u clockfy -n 20 --no-pager; exit 1; }
```

deploy/nginx-clockfy.conf

```
# /etc/nginx/sites-available/clockfy
# Proxy reverso do Clockfy. Troque clockfy.exemplo.com.br pelo seu domínio.
# O HTTPS é adicionado automaticamente pelo certbot (certbot --nginx -d clockfy.exemplo.com.br).
server {
    listen 80;
    listen [::]:80;
    server_name clockfy.exemplo.com.br;

    client_max_body_size 25m;          # recibos e imagens (MAX_UPLOAD_BYTES)
    proxy_read_timeout 300s;          # relatórios grandes e importação

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    gzip on;
    gzip_types text/plain text/css application/json application/javascript image/svg+xml;
}
```

deploy/clockfy.service

```
# /etc/systemd/system/clockfy.service (instalação sem Docker)
[Unit]
Description=Clockfy - controle de horas (clone do Clockify)
After=network.target postgresql.service
Wants=postgresql.service

[Service]
Type=simple
User=clockfy
Group=clockfy
WorkingDirectory=/opt/clockfy/server
EnvironmentFile=-/opt/clockfy/server/.env
Environment=NODE_ENV=production
ExecStart=/usr/bin/node src/index.js
Restart=always
RestartSec=5
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
```

deploy/backup.sh

```
#!/usr/bin/env bash
# Backup diário do banco do Clockfy (Docker ou PostgreSQL local) com retenção e envio opcional ao S3.
# Uso: sudo /opt/clockfy/deploy/backup.sh (agende no cron: 0 3 * * * /opt/clockfy/deploy/backup.sh)
set -euo pipefail
BACKUP_DIR=${BACKUP_DIR:-/var/backups/clockfy}
KEEP_DAYS=${KEEP_DAYS:-14}
S3_BUCKET=${S3_BUCKET:-} # ex.: s3://minha-empresa-backups/clockfy (opcional)
COMPOSE_DIR=${COMPOSE_DIR:-/opt/clockfy} # onde está o docker-compose.yml
STAMP=$(date +%Y%m%d-%H%M%S)
mkdir -p "$BACKUP_DIR"
FILE="$BACKUP_DIR/clockfy-$STAMP.sql.gz"

if docker compose -f "$COMPOSE_DIR/docker-compose.yml" ps db >/dev/null 2>&1; then
    docker compose -f "$COMPOSE_DIR/docker-compose.yml" exec -T db pg_dump -U clockfy -d clockfy | gzip >
"$FILE"
else
    sudo -u postgres pg_dump -d clockfy | gzip > "$FILE"
fi
echo "backup gerado: $FILE ($(du -h "$FILE" | cut -f1))"
find "$BACKUP_DIR" -name 'clockfy-*.sql.gz' -mtime +"$KEEP_DAYS" -delete
if [ -n "$S3_BUCKET" ]; then aws s3 cp "$FILE" "$S3_BUCKET/" && echo "enviado para $S3_BUCKET"; fi
```

.env.example

```
# Copie para .env (na raiz ou em server/) e ajuste.
PORT=3000
NODE_ENV=development
DATABASE_URL=postgres://postgres:postgres@localhost:5432/clockfy
JWT_SECRET=troque-esta-chave
JWT_EXPIRES_IN=30d
APP_URL=http://localhost:3000

# E-mail (opcional - sem SMTP os e-mails são apenas logados no console)
SMTP_HOST=
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=
SMTP_PASS=
SMTP_FROM="Clockfy <no-reply@clockfy.local>"

# Limites e jobs
RATE_LIMIT_PER_SECOND=50
SCHEDULER_ENABLED=true
WEBHOOK_TIMEOUT_MS=10000
MAX_UPLOAD_BYTES=10485760
LOG_SQL=false
```